

LoRA & PEFT Basics

Pre-Work Module 06

ECBS5200 — Practical Deep Learning Engineering

The problem: fine-tuning is expensive

When we fine-tune ModernBERT-base, we update **all 149 million parameters**.

- Every parameter gets a gradient
- Every parameter gets an optimizer state (momentum, variance)
- Every checkpoint saves all 149M parameters
- Training is slow, memory-hungry, and wasteful

What if most of those updates aren't really necessary?

The idea: freeze most, train a few

What if we could:

1. **Freeze** the pretrained weights (don't update them at all)
2. **Add** a small number of new, trainable parameters
3. Get **nearly the same performance** as full fine-tuning

This is the core idea behind **Parameter-Efficient Fine-Tuning (PEFT)**.

LoRA is the most popular PEFT method — and the one we'll use.

LoRA: sticky notes on a textbook

LoRA = Low-Rank Adaptation

Instead of rewriting the textbook, we add **sticky notes**.

- The original weight matrix stays **frozen** (the textbook)
- We add two small matrices that together produce an **adjustment** (the sticky notes)
- During inference: original weight + adjustment = adapted behavior
- The adjustment matrices are **much smaller** than the original

Original: $768 \times 768 = 589,824$ parameters (frozen)
LoRA: $768 \times 8 + 8 \times 768 = 12,288$ parameters (trainable)

That's **48x fewer** trainable parameters per layer.

The numbers

For ModernBERT-base with a typical LoRA configuration:

	Parameters	Optimizer memory
Full fine-tuning	~149M trainable	~1.2 GB
LoRA (r=8)	~1-2M trainable	~16 MB

- **Same base model**, same architecture, same inference
- **Fraction of the training cost**
- Checkpoints go from ~600 MB to ~8 MB

You'll see the exact numbers in the notebook.

Trade-offs

LoRA is not a free lunch. Here's what you're trading:

Advantages:

- Much faster training (fewer parameters to update)
- Much smaller checkpoints (only save the adapters)
- Lower GPU memory requirements
- Can swap adapters for different tasks on the same base model

Potential downsides:

- Might lose a small amount of quality vs. full fine-tuning
- Need to choose rank, target modules, learning rate
- For some tasks, full fine-tuning still wins

Whether the trade-off is worth it depends on your task and constraints.

Week 3 preview

In Week 3, you will:

1. Take the same ModernBERT-base model from Weeks 1-2
2. Apply LoRA adapters to it
3. Fine-tune on our complaint classification task
4. Compare performance and training cost vs. full fine-tuning

For now, just understand the concept:

- Freeze the base model
- Add small trainable matrices
- Train only those

The notebook for this module shows the setup — no training, just the parameter counts.

Key takeaways

1. **Full fine-tuning updates all parameters** — expensive in compute, memory, and storage
2. **PEFT methods freeze most weights** and add a small number of trainable ones
3. **LoRA adds low-rank matrices** alongside frozen weight matrices — like sticky notes on a textbook
4. **The parameter reduction is dramatic** — from ~149M to ~1-2M trainable parameters
5. **Trade-offs exist** — slightly less quality is possible, but often negligible
6. **You'll implement this in Week 3** — for now, understand the concept and see the numbers

Next: run the notebook and see the parameter counts yourself!